Neural Networks via Transfer Learning

Part 1: The Refresher

A quick look at standard Neural Network architectures before we transfer knowledge.

Core Components of NNs



Neurons & Layers

The fundamental units of computation. Arranged in Input, Hidden, and Output layers to process hierarchical representations of data.



Weights & Biases

The learnable parameters. They dictate the strength of connections and shift activation thresholds to map inputs to accurate outputs.



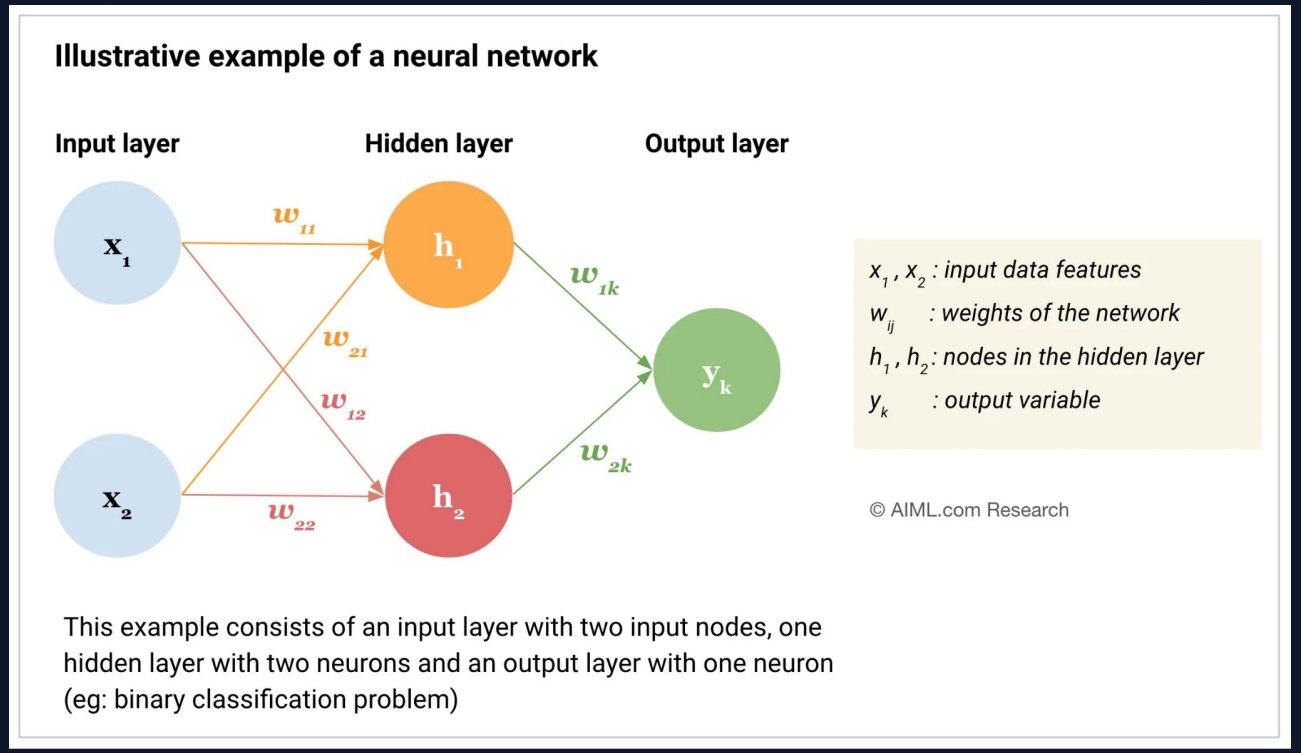
Activations

Non-linear functions (like ReLU, Sigmoid) that allow the network to learn complex, non-linear relationships in real-world data.

How Standard NNs Learn

Training from scratch involves initialising weights randomly and updating them iteratively:

- **Forward Pass:** Data flows through the network to generate a prediction.
- **Loss Calculation:** The prediction is compared to the ground truth using a Loss Function.
- **Backpropagation:** Gradients of the loss with respect to each weight are calculated backwards.
- **Optimisation:** Gradient Descent (e.g., Adam, SGD) updates the weights to minimise future loss.



Architectures & Use Cases



Convolutional (CNN)

Visual data feature extraction.



Recurrent (RNN)

Sequential time-series forecasting.



Transformers

Advanced natural language processing.



Feedforward (FNN)

Tabular classification architectures.

The Cost of Training from Scratch

10M+

Typical Parameter Count

Why Starting Over is Expensive

Modern architectures like ResNet, VGG, or Transformers contain tens to hundreds of millions of parameters.

Training them from absolute zero requires:

- Massive, annotated datasets (e.g., ImageNet with 1.2M+ images).
- Weeks of continuous training on high-end GPU clusters.
- High risk of overfitting if the dataset isn't perfectly balanced and exceptionally large.

Part 2: Enter Transfer Learning

Leveraging existing knowledge to solve new problems efficiently.

A Paradigm Shift

> "Transfer learning will be the next driver of ML commercial success after supervised learning."
>
> — Andrew Ng, Machine Learning Pioneer

Traditional ML vs. Transfer Learning

Traditional Machine Learning

Learning is completely isolated. A model trained to recognize cars starts with zero knowledge when asked to recognize trucks. It requires a massive new dataset and full retraining from a random state.

Result: High data dependency, high compute cost, slower deployment cycles.

Transfer Learning

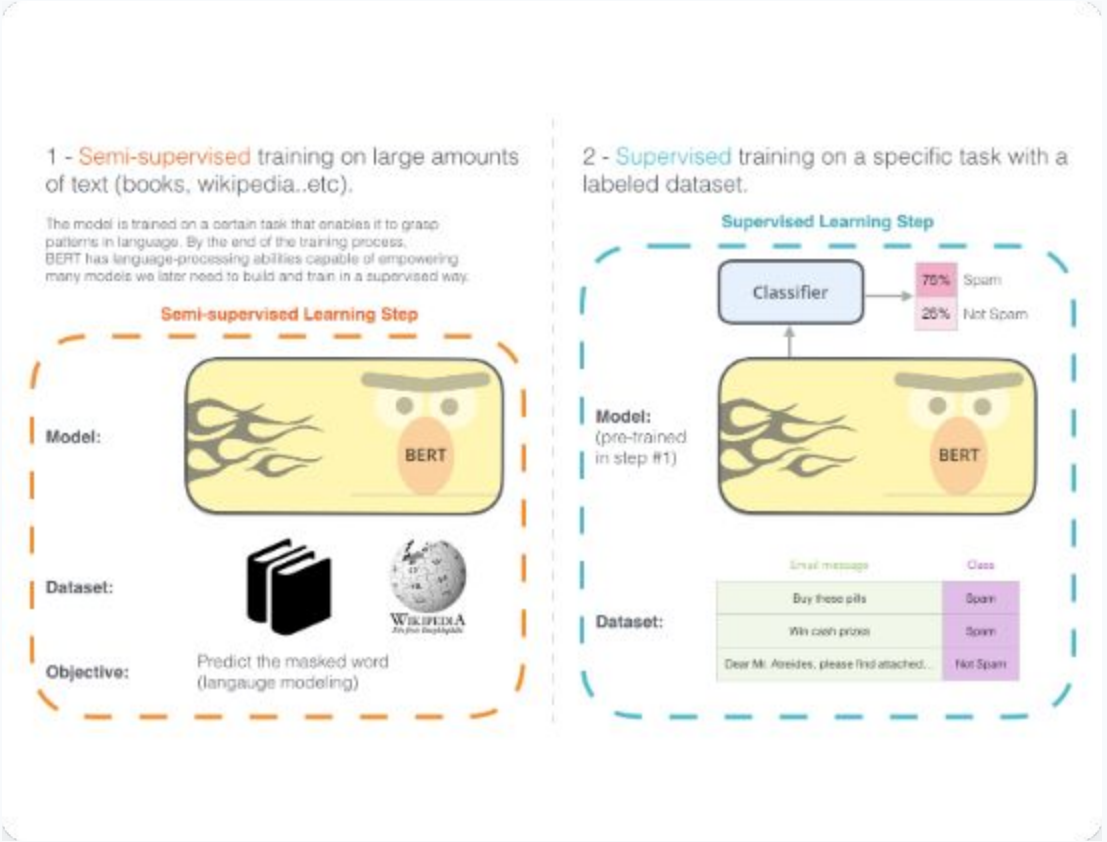
Knowledge is retained. A model trained on millions of generic images already understands edges, shapes, and textures. We transfer this "world view" to a new, specific task (like medical scans).

Result: Less data needed, faster convergence, higher accuracy on small datasets.

The Intuition Behind the Transfer

Transfer learning mimics human learning. If you know how to ride a bicycle, learning to ride a motorcycle is much easier than starting from scratch.

- We take a model trained on a **Source Task** (e.g., general object recognition).
- We extract its learned weights and architecture.
- We apply it to our **Target Task** (e.g., identifying specific dog breeds).
- The model already knows how to "see"; it just needs to learn the new specific classes.



Part 3: Core Concepts

Understanding the vocabulary and mechanisms of knowledge transfer.

Key Terminology



Source Domain

The original environment where the model was initially trained. Typically characterised by massive datasets (e.g., Wikipedia for NLP, ImageNet for Vision) and vast compute resources.



Target Domain

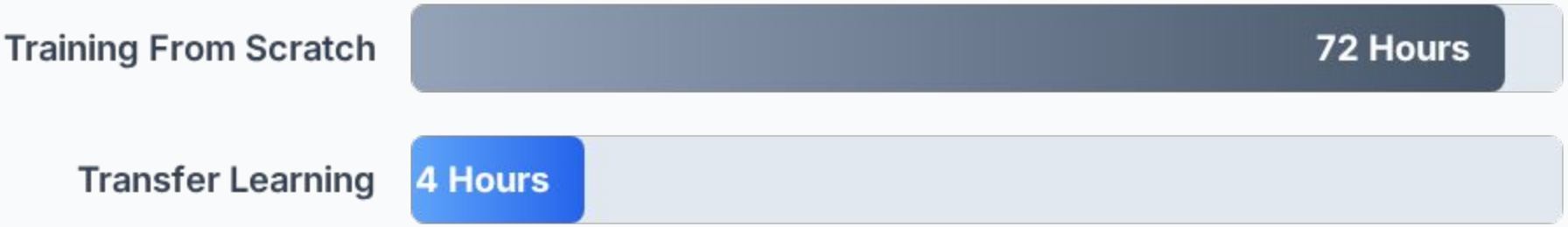
Your specific problem space. This domain usually has far less labelled data available and requires specialised classification based on the general features learned from the Source Domain.



Base Model

The pre-trained architecture (e.g., ResNet50, BERT). We usually strip away its final classification layer and replace it with a new one suited for the Target Domain.

Time to Convergence: Scratch vs Transfer



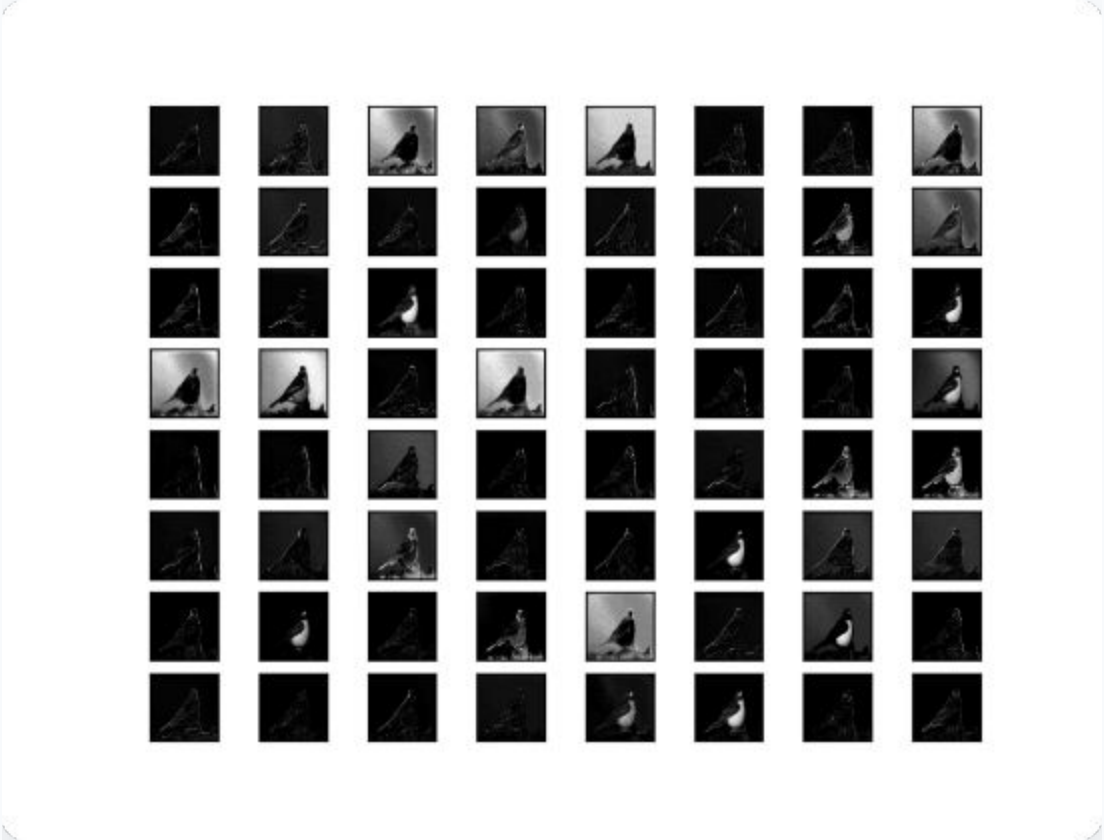
Transfer learning allows models to start closer to the global minimum of the loss function, drastically reducing the number of epochs required to achieve high accuracy.

What Exactly is Transferred?

In Deep Convolutional Neural Networks (CNNs), layers learn hierarchical features:

- **Early Layers:** Learn generic, low-level features like edges, gradients, and basic color blobs. (Highly transferable).
- **Middle Layers:** Combine edges into basic shapes, patterns, and textures.
- **Deep Layers:** Learn highly task-specific features (e.g., dog faces, car wheels). (Less transferable to vastly different tasks).

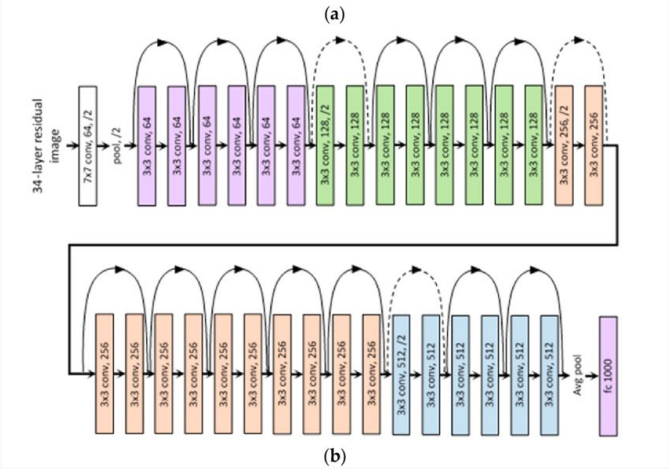
Transfer learning capitalizes heavily on reusing those robust early and middle layer feature extractors.



When to Use Transfer Learning

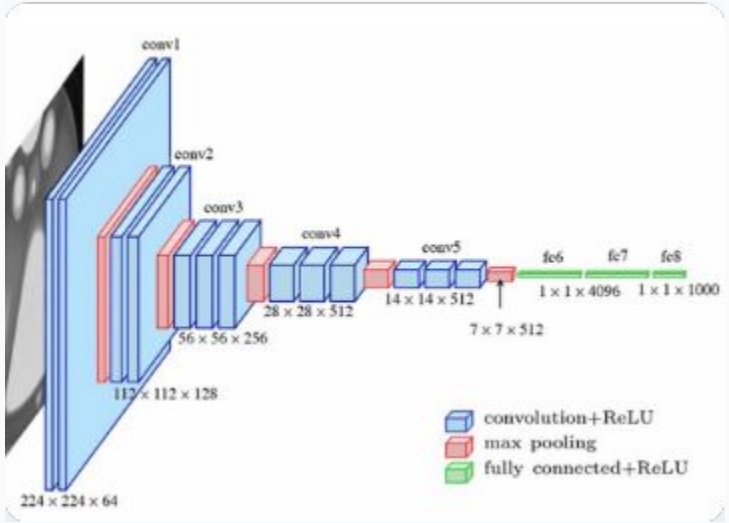
- ✓ **Limited Target Data:** You have only hundreds or thousands of labeled examples, making training from scratch prone to severe overfitting.
- ✓ **Compute Constraints:** You lack the GPU clusters or time budget required to train a deep network over weeks.
- ✓ **Similar Domains:** The source task and target task share underlying data modalities (e.g., transferring from standard photography to satellite imagery).
- ✗ **When NOT to use:** If your target domain is fundamentally completely different from the source domain (e.g., transferring from audio processing to medical image analysis).

Titans of Transfer Learning



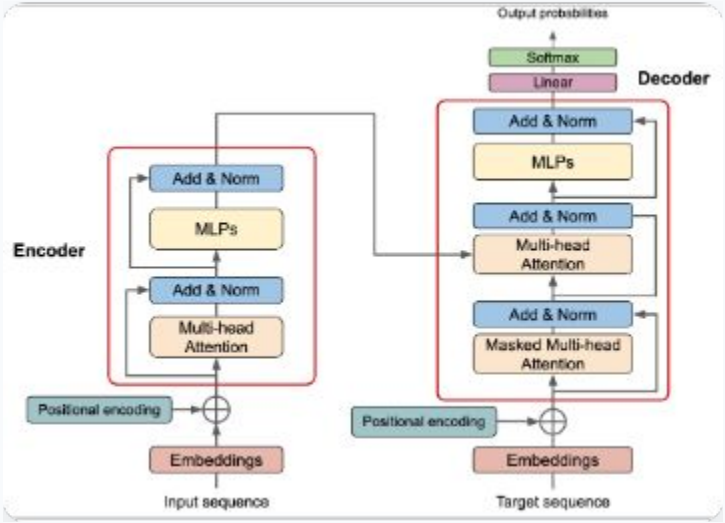
ResNet (Residual Networks)

Introduced skip connections to solve the vanishing gradient problem. The standard for robust image feature extraction.



VGG Series

Known for its elegant, uniform architecture using 3×3 convolutions. Excellent for extracting stylistic features.



Transformers (BERT/GPT)

Revolutionized NLP. Pre-trained on massive text corpora, they provide deep contextual understanding for language tasks.

Part 4: Implementation Strategies

How do we actually perform the transfer in code?

The Two Main Approaches

1. Feature Extraction

The pre-trained model is treated as a static feature extractor. We freeze all the weights of the convolutional base.

We only add a new, untrained dense (fully connected) layer at the top and train *only* those new weights.

Best when: Target dataset is very small.

2. Fine-Tuning

We unfreeze some (or all) of the top layers of the pre-trained base and jointly train both the newly added classifier and the unfreezed base layers.

This allows the model to adjust its higher-order feature representations specifically for the new task.

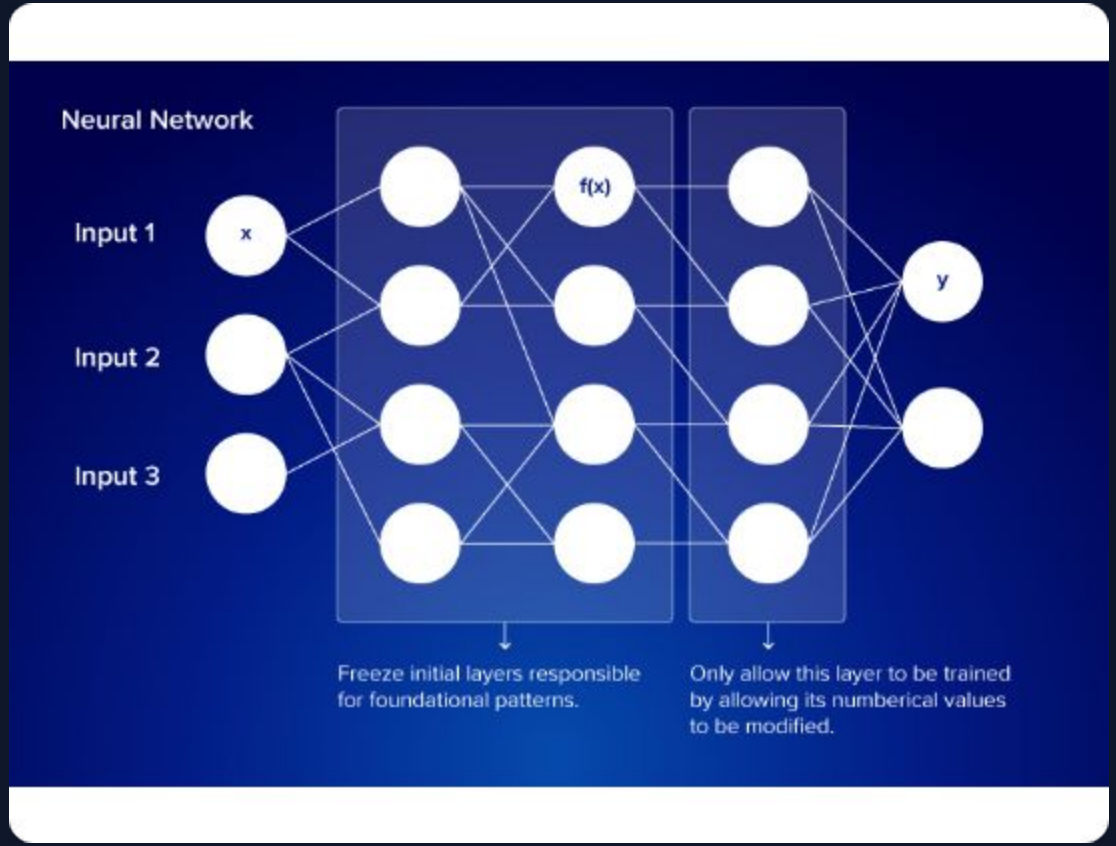
Best when: Target dataset is large and similar.

Deep Dive: Feature Extraction

In this strategy, the base network's weights are "frozen" (layer.trainable = False).

- The frozen base acts as an incredibly complex, non-linear function that turns raw pixels into a dense vector (features).
- These features are passed to a newly initialized dense layer.
- Because the massive base isn't updating, training is remarkably fast and requires minimal VRAM.

It prevents overfitting on small datasets because the powerful base cannot memorize the new data.



The Decision Matrix

Size

vs. Similarity

How to choose your strategy:

- **Small Data, Similar Task:** Feature Extraction. Target data is too small to fine-tune without overfitting.
- **Large Data, Similar Task:** Fine-Tuning. Enough data to adjust weights without overfitting.
- **Small Data, Different Task:** Hardest scenario. Extract features from earlier layers (not deep layers).
- **Large Data, Different Task:** Fine-tune across the entire network, or train from scratch.

Lab Session 1

Applying Feature Extraction in Practice

Lab 1: Image Classification via Feature Extraction

Part 5: Deep Dive into Fine-Tuning

Unlocking the full potential of pre-trained models.

Fine-Tuning Best Practices



Micro Learning Rates

Use an extremely small learning rate (e.g., 1e-5). Large weight updates will destroy the carefully learned features of the pre-trained base.



Top-Down Unfreezing

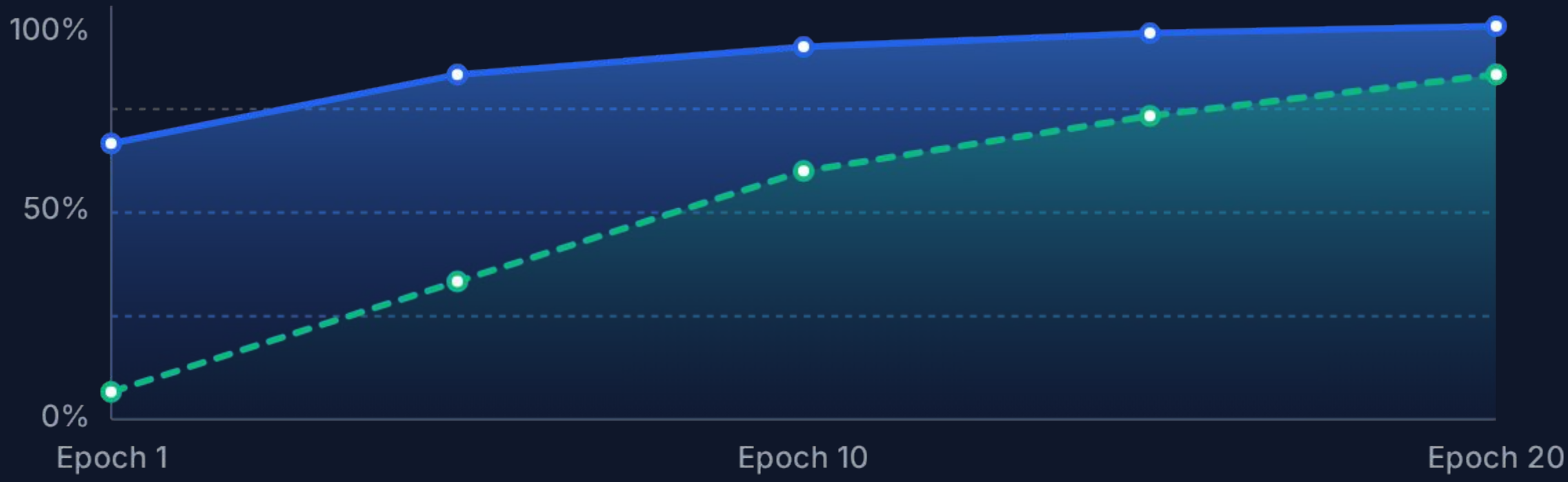
Only unfreeze the top few layers first. Earlier layers contain generic features you likely want to keep static.



Warm-up Phase

Always train the new classifier head (Feature Extraction) fully BEFORE unfreezing the base layers for fine-tuning.

Validation Accuracy over Epochs

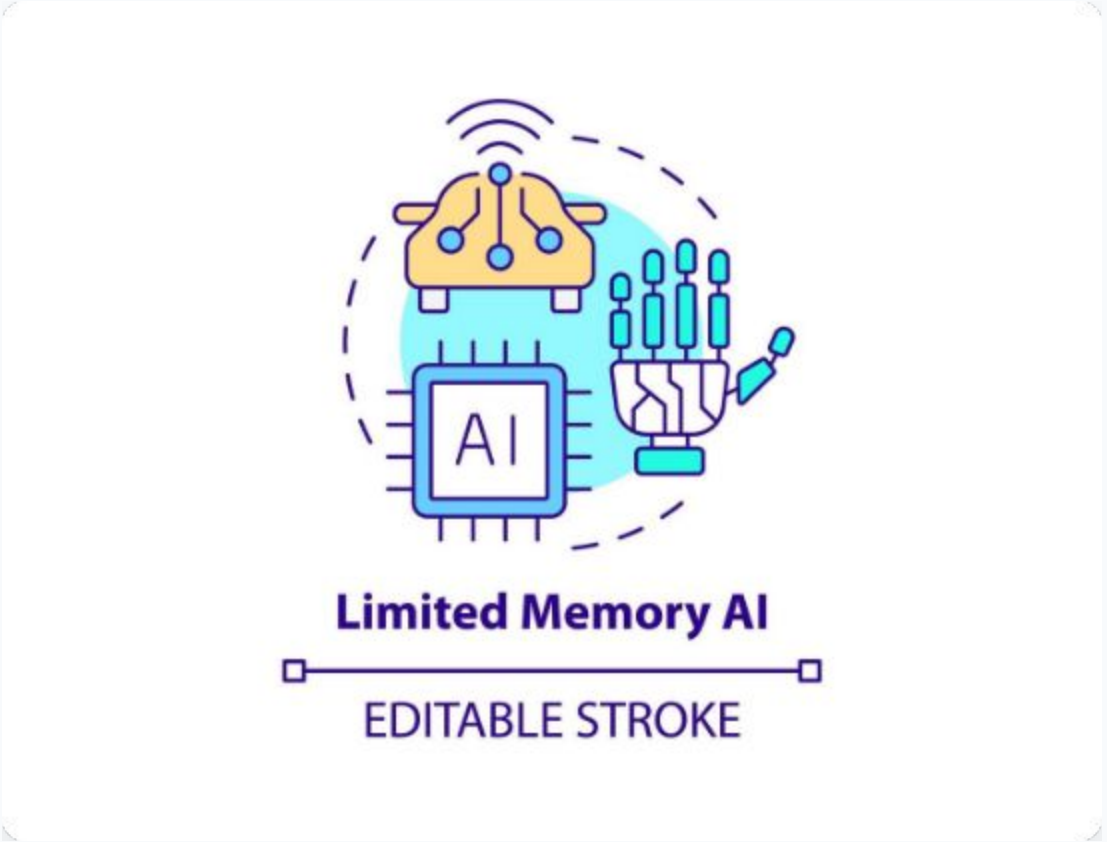


Solid Blue: Transfer Learning starts with high accuracy and converges rapidly. Dashed Green: Training from scratch requires many epochs to catch up.

The Danger: Catastrophic Forgetting

When fine-tuning, the model attempts to minimize loss on the new dataset. If not controlled, it will aggressively overwrite the weights in the pre-trained base.

- The model "forgets" the generalized knowledge it previously had.
- This leads to rapid overfitting on the target dataset.
- It essentially degrades back into a model trained from scratch, negating the benefits of transfer learning.



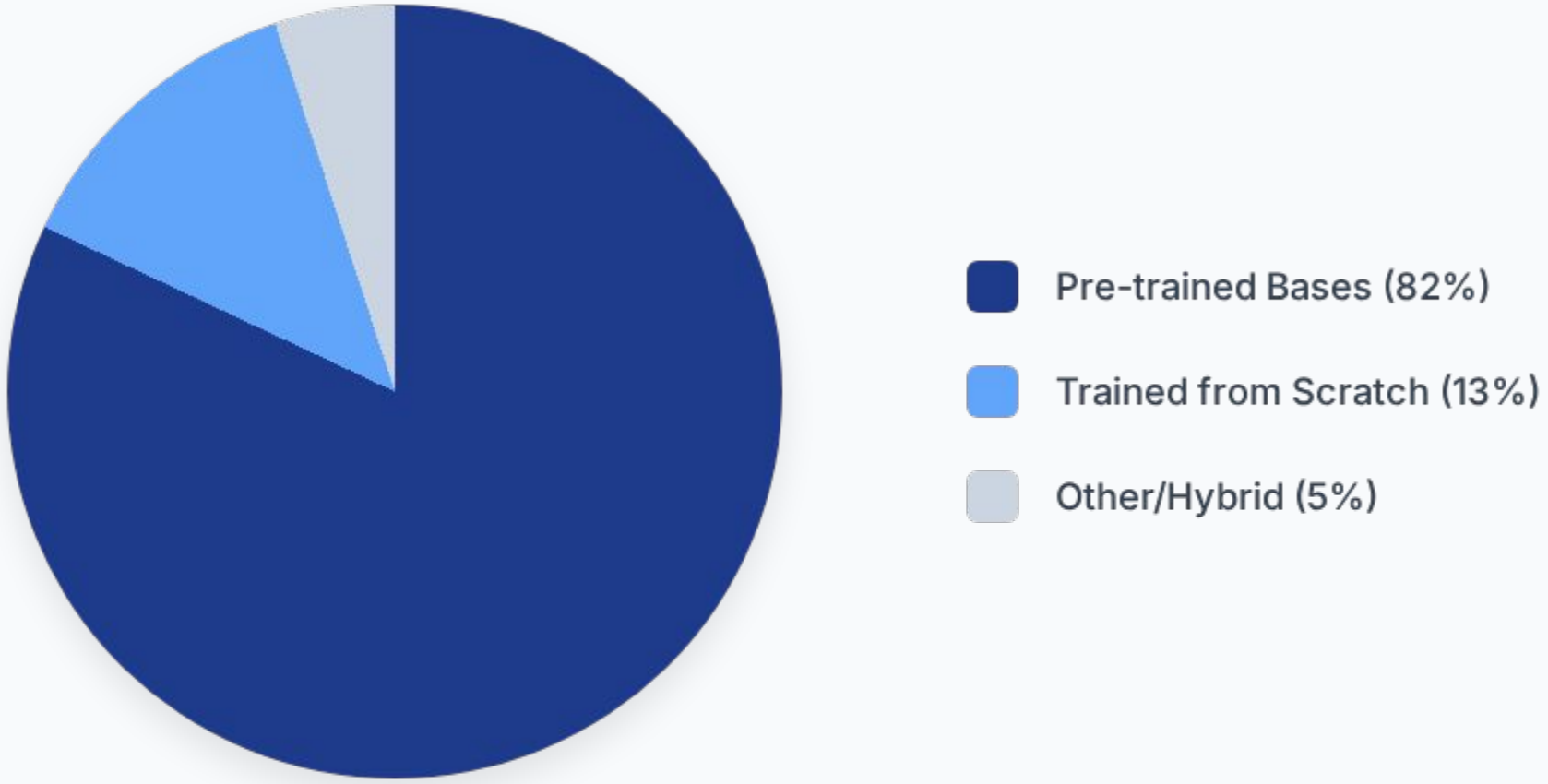
Mitigating Catastrophic Forgetting

- **Gradual Unfreezing:** Unfreeze the network one layer at a time from top to bottom, training briefly after each unfreeze. This provides a gentle adjustment.
- **Weight Decay (L2 Regularization):** Penalize large changes to the weights during the fine-tuning process, forcing the model to rely on existing feature representations.
- **Discriminative Learning Rates:** Apply different learning rates to different layers. The newly added head gets a higher rate (10^-3), while deeper base layers get a much lower rate (10^-6).

Part 6: Real-World Impact & Stats

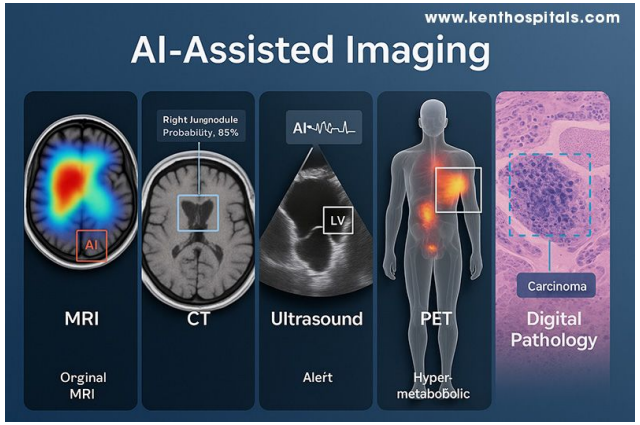
Why industry relies almost exclusively on transfer learning today.

Adoption in Commercial ML Systems



An estimated 82% of commercial NLP and Computer Vision systems deployed today utilize a pre-trained base model rather than training entirely from scratch.

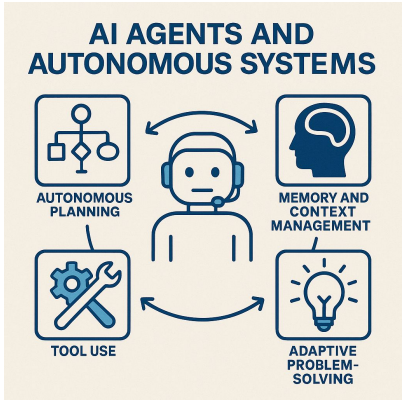
Transfer Learning in Action



HEALTHCARE AI

Medical Imaging

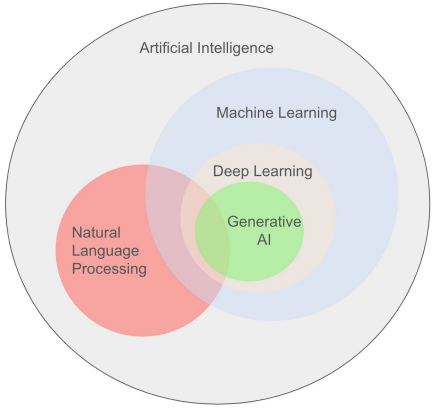
Transferring general vision knowledge to detect tumors in X-rays where labeled datasets are incredibly scarce.



ROBOTICS & MOBILITY

Autonomous Systems

Sim-to-real transfer: Training self-driving agents in virtual simulation environments and transferring the policy to physical cars.



LANGUAGE & GEN AI

Generative AI & NLP

LLMs like GPT-4 are massive foundation models fine-tuned for specific tasks like coding, summarizing, or acting as customer service agents.

Lab 2: End-to-End Fine-Tuning of a CNN